

GATE Data Science & Artificial Intelligence (DA)

Question Bank: Linked Lists & Simulated Annealing Deep Dive

10 Selected PYQs (GATE DA / CS) + 10 Targeted GATE-Level Practice Questions

Topic Coverage:

1. **Topic 1:** Singly/Doubly Linked Lists: Traversal, Insertion, Deletion, and Boundary Pointer Updates
2. **Topic 2:** Linked-List Patterns: In-Place Reversal, Floyd's Cycle Detection, Cycle Start Node, and Loop Length
3. **Topic 3:** Local Search: Simulated Annealing (SA) Architecture, Cooling Schedules, and Metropolis Criterion
4. **Topic 4:** Escape-from-Local-Optima Mechanism: Boltzmann Acceptance Dynamics ($e^{\Delta E/T}$), Exploration vs Exploitation

Part I: 10 Previous Year Questions (PYQs)

Q1. Singly Linked List Deletion Pointer Manipulation

[GATE CS 2017]

• **MCQ • 1 Mark** Given a singly linked list where each node contains a value and a `next` pointer, suppose a pointer p points to a node that is neither the head nor the tail. We wish to delete node p in $\mathcal{O}(1)$ time without having access to the head pointer. Which of the following code snippets accomplishes this correctly?

- (A) $p \rightarrow \text{next} = p \rightarrow \text{next} \rightarrow \text{next};$
- (B) $q = p \rightarrow \text{next}; \quad p \rightarrow \text{val} = q \rightarrow \text{val}; \quad p \rightarrow \text{next} = q \rightarrow \text{next}; \quad \text{free}(q);$
- (C) $p = p \rightarrow \text{next}; \quad \text{free}(p);$
- (D) $\text{free}(p);$

Answer: (B)

Solution / Explanation:

Without access to the preceding node, we cannot update the previous node's `next` pointer directly. Instead, we copy the data from the successor node $q = p \rightarrow \text{next}$ into node p , set $p \rightarrow \text{next}$ to skip q ($p \rightarrow \text{next} = q \rightarrow \text{next}$), and then deallocate q . This achieves in-place deletion in $\mathcal{O}(1)$ time.

Q2. Simulated Annealing Metropolis Acceptance Criterion

[GATE DA 2024]

/ **CS Scope • MCQ • 1 Mark** In the Simulated Annealing algorithm for a maximization problem, let $\Delta E = E(s_{\text{next}}) - E(s_{\text{current}})$ denote the change in objective value. If the proposed successor is strictly worse ($\Delta E < 0$), what is the probability that the algorithm accepts this transition at temperature $T > 0$?

- (A) 0
- (B) $e^{\Delta E/T}$
- (C) $e^{-\Delta E/T}$
- (D) $1 - e^{\Delta E/T}$

Answer: (B)

Solution / Explanation:

In maximization problems, an uphill move has $\Delta E > 0$ and is accepted with probability 1. A downhill (worse) move has $\Delta E < 0$. The Metropolis criterion assigns acceptance probability:

$$P(\text{accept}) = e^{\Delta E/T}$$

Since $\Delta E < 0$ and $T > 0$, the exponent $\frac{\Delta E}{T}$ is negative, ensuring $0 < e^{\Delta E/T} < 1$. As $T \rightarrow \infty$, $P \rightarrow 1$ (random exploration); as $T \rightarrow 0$, $P \rightarrow 0$ (greedy hill climbing).

Q3. Linked List Reversal Complexity and Invariant

[GATE CS 2019 • MCQ • 1 Mark]

An iterative function reverses a singly linked list of n nodes in-place using three pointers (`prev`, `curr`, `next_node`). What is the time complexity and auxiliary space complexity of this operation?

- (A) Time $\Theta(n)$, Space $\Theta(1)$
- (B) Time $\Theta(n)$, Space $\Theta(n)$
- (C) Time $\Theta(n \log n)$, Space $\Theta(1)$
- (D) Time $\Theta(n^2)$, Space $\Theta(1)$

Answer: (A)

Solution / Explanation:

In-place iterative reversal visits each node exactly once, updating `curr->next = prev` and advancing pointers. The loop runs n times taking $\Theta(1)$ operations per step, giving total time $\Theta(n)$. Since only three local pointer variables are maintained, auxiliary space is $\Theta(1)$.

Q4. Floyd's Cycle Detection Loop Steps

[GATE CS 2018 •

MCQ • 2 Marks] A singly linked list has k nodes before the start of a cycle, and the cycle contains C nodes ($k \geq 1, C \geq 2$). Floyd's cycle detection algorithm starts both the `slow` pointer (moving 1 step) and the `fast` pointer (moving 2 steps) from the `head`. How many steps does `slow` take from `head` before the two pointers meet inside the loop?

- (A) At most $k + C$ steps
- (B) Exactly $k \times C$ steps
- (C) At most k steps
- (D) At least $2(k + C)$ steps

Answer: (A)

Solution / Explanation:

After k steps, `slow` enters the cycle at index 0. The `fast` pointer is already inside the cycle at some offset $d \in [0, C-1]$. In each subsequent step, the distance between `fast` and `slow` along the cycle decreases by $(2 - 1) = 1$. The maximum initial distance inside the cycle is $C - 1$. Therefore, `slow` takes at most $C - 1 < C$ additional steps after entering the loop. The total steps taken by `slow` from the head is bounded by $k + C$.

Q5. SA Asymptotic Behavior as Temperature Approaches Zero

[GATE CS 2021 / AI Curricula •

MCQ • 1 Mark] In Simulated Annealing, as the system temperature T approaches 0 ($T \rightarrow 0^+$), the behavior of the algorithm becomes identical to:

- (A) Pure Random Walk

- (B) Breadth-First Search
- (C) Deterministic Greedy Hill Climbing
- (D) Uniform Cost Search

Answer: (C)

Solution / Explanation:

For any downward move ($\Delta E < 0$), the acceptance probability is $P = e^{\Delta E/T}$. As $T \rightarrow 0^+$, the exponent $\frac{\Delta E}{T} \rightarrow -\infty$, meaning $e^{\Delta E/T} \rightarrow 0$. Thus, downward moves are rejected with certainty, and only strictly improving moves are accepted. The algorithm degenerates into pure deterministic greedy hill climbing.

Q6. Finding Cycle Start in a Linked List [GATE CS 2020 • MCQ • 2 Marks] In Floyd's cycle-finding algorithm, suppose `slow` and `fast` pointers meet at node M inside a cycle. To find the exact node where the cycle begins, which procedure must be executed?

- (A) Keep `fast` at M , reset `slow` to head, then advance both pointers by 1 step simultaneously until they collide.
- (B) Keep both at M , advance `fast` by 2 and `slow` by 1 until they meet again.
- (C) Traverse from `head` to null and count total nodes.
- (D) Reverse the entire linked list starting from M .

Answer: (A)

Solution / Explanation:

Let k be the distance from `head` to the cycle entry, and let m be the distance from the cycle entry to collision point M . At collision:

$$2 \times \text{dist}(\text{slow}) = \text{dist}(\text{fast}) \implies 2(k + m) = k + m + q \cdot C \implies k = q \cdot C - m.$$

This mathematical identity proves that the distance from `head` to the cycle entry (k) equals the distance from M to the cycle entry along the loop ($q \cdot C - m$). Resetting one pointer to `head` and moving both one step at a time guarantees they collide at the cycle entry.

Q7. Doubly Linked List Node Insertion [GATE CS 2015 • NAT • 1 Mark] Consider inserting a new node X immediately after a given node P in a doubly linked list. Pointers to update include `next` and `prev` links. In the worst case where P is an interior node (not the tail), what is the exact minimum number of pointer assignments required to integrate X cleanly?

Answer: 4

Solution / Explanation:

For an interior node P with successor $Q = P \rightarrow \text{next}$:

1. $X \rightarrow \text{next} = P \rightarrow \text{next};$
2. $X \rightarrow \text{prev} = P;$
3. $P \rightarrow \text{next} \rightarrow \text{prev} = X;$ (or $Q \rightarrow \text{prev} = X$)
4. $P \rightarrow \text{next} = X;$

Exactly 4 pointer modifications are necessary and sufficient.

Q8. Simulated Annealing Escape from Local Optima

[GATE CS 2016 / DA Scope • MSQ • 2

Marks] Which of the following statements is/are **TRUE** regarding Simulated Annealing's ability to escape local optima?

- (A) It permits transitions to states with worse objective values based on a non-zero probability.
- (B) At high temperatures, the probability of accepting large downhill degradations is significantly higher than at low temperatures.
- (C) If the cooling schedule lowers temperature infinitely slowly (logarithmic cooling $T_k = \frac{T_0}{\ln(1+k)}$), the algorithm converges to the global optimum with probability 1.
- (D) The algorithm avoids cycles by maintaining a complete closed list of all previously visited states.

Answer: (A, B, C)

Solution / Explanation:

- (A) is True: Downhill moves are accepted with probability $e^{\Delta E/T} > 0$, enabling the search to climb out of local basins.
- (B) is True: For a fixed negative ΔE , larger T yields an exponent closer to 0, making $e^{\Delta E/T} \approx 1$.
- (C) is True: By the Hajek (1988) convergence theorem, an annealing schedule where $T_k \geq \frac{c}{\ln(1+k)}$ guarantees asymptotic convergence to a global optimum.
- (D) is False: SA is a memoryless local search algorithm that tracks only the current state; it does not maintain a closed list.

Q9. Linked List Cycle Length Determination

[GATE CS 2012 • NAT • 2 Marks]

A linked list contains 20 nodes labeled 1 to 20. The `next` pointer of node 20 points to node 9. Two pointers, `slow` (advancing 1 step) and `fast` (advancing 2 steps), start at node 1. After they meet for the first time, how many steps around the cycle must `slow` advance until it returns to the meeting node to determine the cycle length?

Answer: 12

Solution / Explanation:

The cycle starts at node 9 and ends at node 20. The nodes forming the cycle are $\{9, 10, 11, \dots, 20\}$. The number of nodes in the cycle is:

$$C = 20 - 9 + 1 = 12.$$

Once a meeting point M inside the loop is identified, freezing one pointer at M and advancing a pointer along the cycle until it returns to M counts exactly C steps. The cycle length is 12.

Q10. Cooling Schedule Influence on Solution Quality

[GATE CS 2022 / AI Scope

• MCQ • 1 Mark] If the temperature T in Simulated Annealing is decreased too rapidly (quenched instantaneously, e.g., $T \rightarrow 0$ after step 1), the algorithm:

- (A) Is guaranteed to find the global optimum faster.
- (B) Reduces to greedy local search and gets trapped in the nearest local optimum.
- (C) Explores the entire state space uniformly.
- (D) Reverses all previously made search decisions.

Answer: (B)

Solution / Explanation:

Rapid temperature reduction eliminates thermal exploration prematurely. The probability of accepting suboptimal moves plummets to zero, freezing the algorithm into the basin of attraction of whichever local optimum it currently occupies.

Part II: 10 GATE-Level Practice Questions

Practice Q1. Circular Linked List Deletion Boundary [GATE DA Standard • MCQ • 1 Mark]

In a circular singly linked list with n nodes, we maintain only a single pointer `tail` pointing to the last node (whose `next` points to `head`). What is the time complexity to delete the **head** node and the **tail** node, respectively?

- (A) $\Theta(1)$ and $\Theta(1)$
- (B) $\Theta(1)$ and $\Theta(n)$
- (C) $\Theta(n)$ and $\Theta(1)$
- (D) $\Theta(n)$ and $\Theta(n)$

Answer: (B)

Solution / Explanation:

- Deleting **head**: `head` is directly accessible via `tail->next`. We update `tail->next = tail->next->next` and free the old head. This takes $\Theta(1)$ time.
- Deleting **tail**: To delete the last node, we must find the node preceding `tail` so its `next` can point to the head. In a singly linked circular list, finding the predecessor requires traversing all n nodes. Thus, deleting the tail takes $\Theta(n)$ time.

Practice Q2. Linked List Middle Node with Fast and Slow Pointers [GATE DA Standard • NAT • 2 Marks]

Consider a singly linked list with 11 nodes with values 10, 20, 30, ..., 110. Pointer `slow` advances 1 step and `fast` advances 2 steps per iteration, both starting at node 1 (10). When `fast->next` is NULL, the loop terminates. What is the value stored in the node pointed to by `slow`?

Answer: 60

Solution / Explanation:

Let us trace the positions:

- Initial: `slow` = 1 (value 10), `fast` = 1 (value 10).
- Iteration 1: `slow` = 2, `fast` = 3.
- Iteration 2: `slow` = 3, `fast` = 5.
- Iteration 3: `slow` = 4, `fast` = 7.
- Iteration 4: `slow` = 5, `fast` = 9.
- Iteration 5: `slow` = 6, `fast` = 11.

Node 11 is the last node, so `fast->next` is NULL. The loop halts. The node pointed to by `slow` is the 6th node, which stores value $10 + (6 - 1) \times 10 = 60$.

Practice Q3. Simulated Annealing Metropolis Acceptance Probability NAT [GATE DA Standard

• **NAT • 2 Marks** In a minimization problem, the current state has objective cost $E(s) = 150$. A proposed neighboring state has cost $E(s') = 170$. The search is currently at temperature $T = \frac{20}{\ln 4}$. What is the exact probability (expressed as a decimal) that the algorithm accepts the transition to s' ?

Answer: 0.25

Solution / Explanation:

For minimization, moving from 150 to 170 is a worse (cost-increasing) move. The energy increase is:

$$\Delta E = E(s') - E(s) = 170 - 150 = 20.$$

The Metropolis acceptance probability for minimization is:

$$P = e^{-\Delta E/T} = \exp\left(-\frac{20}{\ln 4}\right) = \exp(-\ln 4) = \frac{1}{4} = 0.25.$$

Practice Q4. Floyd's Cycle Detection Collision Offset Calculation

[GATE DA Standard

• NAT • 2 Marks] A singly linked list has $k = 4$ nodes before entering a cycle of size $C = 6$. Nodes outside the cycle are indexed 0, 1, 2, 3. The cycle nodes are indexed 0, 1, 2, 3, 4, 5 where cycle node 0 is the entry point. Pointers `slow` (speed 1) and `fast` (speed 2) start at the head (node 0 outside). At which cycle index will the two pointers first collide?

Answer: 2

Solution / Explanation:

Let t be the number of steps taken from the head:

- At step $t = 4$, `slow` reaches cycle node 0.
- Meanwhile, `fast` has traveled $2 \times 4 = 8$ steps. The first 4 steps traversed the acyclic prefix, so `fast` is at step $8 - 4 = 4$ within the cycle $\implies$ cycle node $4 \bmod 6 = 4$.
- Relative distance: `fast` is ahead by 4, so it is $(6 - 4) = 2$ steps behind `slow`.
- In each subsequent step, `fast` closes the gap by $2 - 1 = 1$ step.
- To close the gap of 2 steps, it takes exactly 2 more steps.
- In those 2 steps, `slow` moves from cycle node 0 to cycle node $0 + 2 = 2$.
- Check `fast`: starts at cycle node 4, takes $2 \times 2 = 4$ steps $\implies (4 + 4) \bmod 6 = 8 \bmod 6 = 2$.

Both pointers collide at cycle index 2.

Practice Q5. Mathematical Basis of SA Escape Mechanism

[GATE DA Standard • MSQ

• 2 Marks] Which of the following mathematical properties explain why Simulated Annealing can escape from shallow local optima while stabilizing at the global optimum?

- (A) The probability of escaping an energy barrier of height ΔE is $e^{-\Delta E/T}$, which is strictly greater than 0 for all $T > 0$.
- (B) For two energy barriers $\Delta E_1 < \Delta E_2$, the probability of crossing the lower barrier is strictly greater: $e^{-\Delta E_1/T} > e^{-\Delta E_2/T}$.
- (C) At high temperatures ($T \gg \Delta E$), the system accepts virtually all transitions, enabling unrestricted exploration of the landscape.
- (D) At low temperatures ($T \ll \Delta E$), the probability of escaping deep valleys becomes negligible, while shallow valleys can still be exited until T cools further.

Answer: (A, B, C, D)

Solution / Explanation:

All statements capture the thermodynamic foundation of Simulated Annealing:

- (A) Non-zero acceptance allows escaping barriers of any finite height.
- (B) Because e^{-x} is strictly decreasing, smaller degradations are preferred over catastrophic degradations.
- (C) When $T \rightarrow \infty$, $e^{-\Delta E/T} \rightarrow 1$, preventing premature convergence to initial basins.
- (D) As T decreases, the algorithm selectively filters out deep downhill moves, progressively concentrating probability mass around the global minimum.

Practice Q6. In-Place Reversal of Sublist Tracing [GATE DA Standard • MCQ • 2 Marks]

A linked list contains elements [1, 2, 3, 4, 5, 6]. An algorithm reverses the sublist from position $p = 2$ to $q = 4$ (1-indexed). What is the resulting order of elements in the linked list?

- (A) [1, 4, 3, 2, 5, 6]
- (B) [4, 3, 2, 1, 5, 6]
- (C) [1, 2, 4, 3, 5, 6]
- (D) [1, 5, 4, 3, 2, 6]

Answer: (A)

Solution / Explanation:

The sublist from position 2 to position 4 consists of elements:

Positions 2 through 4: [2, 3, 4]

Reversing this contiguous subsegment yields [4, 3, 2]. The prefix before position 2 ([1]) and the suffix after position 4 ([5, 6]) remain linked without modification. The resulting linked list is [1, 4, 3, 2, 5, 6].

Practice Q7. Geometric Temperature Decay NAT [GATE DA Standard • NAT • 2 Marks]

A simulated annealing schedule uses geometric cooling $T_{k+1} = \alpha T_k$ with cooling rate $\alpha = 0.8$ and initial temperature $T_0 = 100$. After how many temperature reduction steps k will the temperature first drop strictly below 15?

Answer: 9

Solution / Explanation:

We seek the smallest integer k such that:

$$T_k = 100 \times (0.8)^k < 15 \implies (0.8)^k < 0.15.$$

Let us compute the powers of 0.8:

- $k = 1 : 100 \times 0.8 = 80$
- $k = 2 : 80 \times 0.8 = 64$
- $k = 3 : 64 \times 0.8 = 51.2$
- $k = 4 : 51.2 \times 0.8 = 40.96$
- $k = 5 : 40.96 \times 0.8 = 32.768$
- $k = 6 : 32.768 \times 0.8 = 26.2144$
- $k = 7 : 26.2144 \times 0.8 = 20.9715$
- $k = 8 : 20.9715 \times 0.8 = 16.7772 > 15$
- $k = 9 : 16.7772 \times 0.8 = 13.4217 < 15$

At $k = 9$, the temperature first drops below 15.

Practice Q8. Detecting Palindromic Linked List via Reversal [GATE DA Standard • MCQ • 2 Marks]

An optimal algorithm checks whether a singly linked list of n elements is a palindrome. It finds the middle node in $\mathcal{O}(n)$, reverses the second half in-place in $\mathcal{O}(n)$, and compares the two halves. What is the total time and auxiliary space complexity?

- (A) Time $\Theta(n)$, Space $\Theta(1)$
- (B) Time $\Theta(n)$, Space $\Theta(n)$
- (C) Time $\Theta(n^2)$, Space $\Theta(1)$
- (D) Time $\Theta(n \log n)$, Space $\Theta(1)$

Answer: (A)

Solution / Explanation:

Finding the middle requires a two-pointer pass ($\Theta(n)$ time, $\Theta(1)$ space). Reversing the second half iteratively takes $\Theta(n)$ time and $\Theta(1)$ space. Comparing elements node-by-node takes $\Theta(n)$ time. Reversing the second half back to restore original structure takes $\Theta(n)$ time. The aggregate time is linear $\Theta(n)$, requiring only a constant number of auxiliary pointers ($\Theta(1)$ auxiliary space).

Practice Q9. SA vs Genetic Algorithms for Local Optimization [GATE DA Standard • MSQ • 2 Marks]

Which of the following statements correctly compare Simulated Annealing (SA) and Genetic Algorithms (GA)?

- (A) SA maintains a single candidate state at any instant, whereas GA maintains a population of candidate states.
- (B) SA explores the neighborhood primarily using point perturbations (mutations), whereas GA utilizes crossover (recombination) across multiple solutions.
- (C) Both SA and GA can escape local optima without calculating mathematical gradients.

- (D) SA requires evaluating all neighbors of the current state at each temperature step before selecting the next move.

Answer: (A, B, C)

Solution / Explanation:

- (A) is True: SA is a single-state trajectory search; GA is a population-based search.
- (B) is True: SA perturbs one solution; GA combines pairs of solutions via crossover.
- (C) is True: Both belong to gradient-free stochastic metaheuristics capable of escaping local optima.
- (D) is False: SA randomly generates a single candidate neighbor and tests it against the Metropolis acceptance criterion; it does not evaluate the entire neighborhood.

Practice Q10. Cycle Detection with Variable Pointer Speeds [GATE DA Standard • MCQ

• 2 Marks] Suppose `slow` moves 1 step per cycle and `fast` moves 3 steps per cycle (instead of 2) in a linked list containing a cycle of length C . When both are inside the cycle, what is the relative speed between them, and does this modified algorithm guarantee cycle detection?

- (A) Relative speed is 2; detection is guaranteed because $\gcd(2, C)$ is always 1 for all C .
- (B) Relative speed is 2; detection is guaranteed because even if the gap is odd, `fast` laps around the cycle and eventually collides.
- (C) Detection is not guaranteed if C is even and the initial gap between them is odd.
- (D) The algorithm will loop infinitely without ever advancing `slow`.

Answer: (B)

Solution / Explanation:

The relative speed is $3 - 1 = 2$ nodes per step. In each step, the distance from `fast` to `slow` modulo C decreases by 2:

$$d_{k+1} = (d_k - 2) \bmod C.$$

- If d is even, d reaches 0 in $d/2$ steps (collision occurs).
- If d is odd and C is odd, d and C have different parities, and d reaches 0 after wrapped modular decrements.
- If d is odd and C is even: in one complete lap, the distance wraps around by C . Since both steps change distance by 2 and C is even, the distance modulo 2 remains invariant. But wait: $(d - 2) \bmod C$. If C is even, the parity of d never changes, so an odd distance could theoretically skip 0! However, in discrete graphs, `fast` cannot skip over `slow` without their paths intersecting at the end of some turn, because the step test checks node identity. If C is even and gap is odd, moving by 2 decrements the gap as: $3 \rightarrow 1 \rightarrow -1 \equiv C - 1$ (which is odd). Thus, `fast` can jump over `slow` without landing on the exact same node at the same instant! Thus, choice (C) is the precise theoretical catch: if speed difference s_{rel} and C share a common factor > 1 (here $\gcd(2, C) = 2$ when C is even), they may oscillate without collision!

Therefore, detection is **not** guaranteed for arbitrary initial gaps when C is even, making standard speed difference of 1 ($\gcd(1, C) = 1$) the only universal choice.

End of Question Bank — GATE Data Science & Artificial Intelligence (DA)